

India Learns LMS — Faculty Section

Developer Handoff Specification Companion document to the interactive prototype

((india_learns_faculty_lms.jsx)) Version 0.1 · April 24, 2026

1. Purpose & Scope

The India Learns LMS is the organizational and quality-assurance backbone for in-person classes. All course material, assignments, grading, feedback, and attendance must flow through the platform. Faculty is the primary user of this section; super admins have equivalent structural permissions plus global oversight. Students consume the outputs through a separate student UI that is out of scope for this document.

The current production UI presents a course as a single vertical stream of Announcements and Assignments, which does not reflect the shape of a course as it is actually taught. This redesign introduces a **Course → Module → Session** hierarchy and makes the session the primary container for materials, assignments, and attendance.

Scope covered here: the faculty workspace — dashboard, course shell with six tabs (Overview, Content, Gradebook, Students, Announcements, Settings), and the session detail view. Students, Announcements, and Settings tabs are structurally scoped but not designed in visual detail in v0.1; their data models are defined so implementation can proceed without blocking.

2. Architecture Overview

The prototype is a single-page React application. Recommended production stack: React 18+ with TypeScript, React Router v6 for routing, TanStack Query for server state, and Tailwind CSS for styling. The existing prototype uses inline styles with Tailwind utilities that render without a build step; the first development task should be to extract tokens into a proper Tailwind config (see §6).

State management should stay deliberately simple. Server data lives in React Query caches. Local UI state (expanded modules, active tab, gradebook view mode) lives in component state or URL query parameters. Do not introduce Redux or Zustand unless a concrete cross-screen state need emerges — none exists in the current scope.

All write actions should follow one of two patterns depending on reversibility. Low-risk actions (expand/collapse, reorder materials, save draft feedback) use optimistic updates with silent rollback. High-risk actions (publish grade to student, delete session, mark session complete, post announcement) require explicit user confirmation and server acknowledgment before the UI reflects the change.

3. Information Architecture & Routes

The workspace has two levels of navigation. The **global left nav** is persistent and contains top-level destinations. The **course header tabs** appear only inside a course and switch between facets of the same course.

/	→ redirects to /dashboard
/dashboard	→ cross-course landing page
/courses	→ list of all courses taught by this faculty
/courses/:courseId	→ redirects to
/courses/:courseId/content	
/courses/:courseId/overview	→ QA & progress view
/courses/:courseId/content tab)	→ Module → Session tree (default
/courses/:courseId/sessions/:sessionId	→ session detail page
/courses/:courseId/gradebook	→ grid view (default)
/courses/:courseId/gradebook/assignments/:id	→ per-assignment view
/courses/:courseId/students	→ roster
/courses/:courseId/students/:studentId	→ individual student profile
/courses/:courseId/announcements	→ announcement composer + history
/courses/:courseId/settings	→ course metadata and policies
/grading	→ cross-course "to grade" inbox
/timetable	→ weekly schedule
/profile	
/notifications	

The prototype's `activeScreen` / `activeTab` / `activeSession` state should be replaced with URL-driven routing. This enables deep-linking from email notifications, browser back-button behavior, and sharing links to specific sessions or assignments with colleagues.

4. Data Model

The following entities are required. Types are in TypeScript; translate to your chosen backend ORM.

ts

```

type UserRole = 'faculty' | 'super_admin' | 'student';

interface User {
  id: string;
  name: string;
  email: string;
  role: UserRole;
  avatarUrl?: string;
}

interface Course {
  id: string;
  code: string; // e.g. "DAAM 101"
  title: string;
  term: string; // e.g. "Spring 2026"
  description?: string;
  totalHours?: number; // from curriculum generator
  contactHours?: number;
  independentHours?: number;
  facultyIds: string[];
  studentIds: string[];
  createdAt: string;
  archivedAt?: string;

  // Curriculum generator link – see §9
  sourceWorkflowId?: string; // generator workflow ID
  sourceWorkflowVersion?: string; // schema version at import time
  sourceDocument?: string; // .docx reference for fallback import path
  lastSyncedAt?: string; // for future re-sync (Option C in §9.5)
  programLearningOutcomes?: ProgramLearningOutcome[]; // from generator Step 3
}

interface ProgramLearningOutcome {
  id: string;
  number: number;
  statement: string;
  bloomLevel: string;
}

interface Module {
  id: string;
  courseId: string;
  code?: string; // e.g. "HRM101" (from generator)
}

```

```

title: string;
order: number; // drives display sequence
description?: string;
aim?: string; // one-sentence module aim (from generator Step 4)
coreElective?: 'core' | 'elective';
prerequisites?: string[]; // module codes this module depends on
learningOutcomes: ModuleLearningOutcome[];
totalHours?: number;
contactHours?: number;
independentHours?: number;
}

interface ModuleLearningOutcome {
  id: string; // e.g. "M1-L01"
  number: number;
  description: string;
  bloomLevel: 'remember' | 'understand' | 'apply' | 'analyse' | 'evaluate' | 'create';
  linkedPLOs?: string[]; // Program Learning Outcome IDs
  linkedKSCs?: string[]; // KSC codes: K1, S3, C2, etc.
}

interface Session {
  id: string;
  moduleId: string;
  courseId: string; // denormalized for query performance
  number: number; // sequential within the course
  title: string;
  description?: string;
  type?: 'lecture' | 'seminar' | 'workshop' | 'tutorial' | 'lab';
  plannedMinutes?: number; // from generator Step 10 (lesson duration)
  scheduledStart: string; // ISO datetime
  scheduledEnd: string;
  location?: string;
  status: 'upcoming' | 'in_progress' | 'completed';
  completedAt?: string;
  completedBy?: string;
  notes?: string; // private: faculty + super_admin only

  // Curriculum generator fields
  sourceLessonId?: string; // original lesson ID in generator
  linkedMLOs?: string[]; // MLO IDs this session targets
  activities?: SessionActivity[]; // structured activity sequence from Step 10
  formativeChecks?: FormativeCheck[]; // quick checks from Step 10
  objectives?: string[]; // from generator, also rendered in description

```

```

}

interface SessionActivity {
  id: string;
  order: number;
  type: 'mini_lecture' | 'discussion' | 'demonstration' | 'practice'
    | 'role_play' | 'case_analysis' | 'group_work' | 'assessment' | 'break';
  title: string;
  description: string;
  durationMinutes: number;
  teachingMethod?: string;
  resources?: string[];
  instructorActions?: string[];
  studentActions?: string[];
}

interface FormativeCheck {
  id: string;
  type: 'mcq' | 'quick_poll' | 'discussion_question' | 'reflection';
  question: string;
  options?: string[];
  correctAnswer?: string;
  explanation?: string;
  linkedMLO?: string;
  durationMinutes: number;
}

interface Material {
  id: string;
  sessionId?: string;           // attached to session (Content tab)
  assignmentId?: string;       // OR attached to assignment (supporting files)
  moduleId?: string;           // OR module-level resource
  type: 'slides' | 'pdf' | 'video' | 'link' | 'reading' | 'practice' | 'reflection'
  title: string;
  url: string;
  sizeBytes?: number;
  expectedHours?: number;      // for independent activities
  uploadedBy: string;
  uploadedAt: string;
}

interface Assignment {
  id: string;
  sessionId: string;           // required – every assignment ties to a session

```

```

courseId: string;
title: string;
description: string;
type?: 'assignment' | 'exam' | 'case_study';
dueAt: string;
maxScore: number;
rubric?: RubricCriterion[];
attachments?: Material[]; // supporting material students need
expectedHours?: number; // from curriculum generator, if imported

// Curriculum generator fields
sourceAssignmentPackId?: string; // original ID in generator Step 12
linkedMLOs?: string[]; // MLO IDs this assignment assesses
submissionRequirements?: string[]; // evidence requirements from Step 12
sectionBreakdown?: ExamSection[]; // for summative exams from Step 13

createdBy: string;
createdAt: string;
}

interface ExamSection {
  section: 'A' | 'B' | 'C';
  title: string;
  type: 'mcq' | 'scenario' | 'case';
  weight: number; // percentage of total
  questionCount: number;
}

interface Submission {
  id: string;
  assignmentId: string;
  studentId: string;
  submittedAt?: string; // null = not submitted
  files: SubmissionFile[];
  status: 'missing' | 'submitted' | 'needs_grading' | 'graded';
  grade?: number;
  feedback?: string;
  gradedBy?: string;
  gradedAt?: string;
  publishedToStudent: boolean; // two-step grade release
}

interface AttendanceRecord {
  id: string;

```

```

sessionId: string;
studentId: string;
status: 'present' | 'absent' | 'late';
markedBy: string;
markedAt: string;
}

interface Announcement {
  id: string;
  courseId: string;
  subject: string;
  body: string;
  postedBy: string;
  postedAt: string;
  readBy: string[];
}

```

Relationships to enforce at the data layer: A Session always belongs to a Module; a Module always belongs to a Course. An Assignment must reference a Session — there is no course-level or module-level assignment. A Material can be attached at Session, Assignment, or Module scope but exactly one of those must be set.

Computed fields are not stored; calculate on read. Course progress = completed sessions ÷ total sessions. Course attendance rate = sum of `present` records ÷ total possible attendance records (students × held sessions). Grading backlog for a course = submissions with status `needs_grading` tied to that course. Student course grade = average of graded submissions for that student in that course, weighted by `maxScore` if weighting is introduced later.

5. Permission Model

Action	Faculty (assigned to course)	Super Admin	Student
View course content	✓	✓ (any course)	✓ (enrolled only)
Create / edit / delete modules	✓	✓	✗
Create / edit / delete sessions	✓	✓	✗
Upload / delete materials	✓	✓	✗
Create / edit / delete assignments	✓	✓	✗

Action	Faculty (assigned to course)	Super Admin	Student
Submit assignments	X	X	✓
Grade submissions	✓	✓	X
Mark attendance	✓	✓	X
Post announcements	✓	✓	X
Edit course metadata	✓	✓	X
View private session notes	✓	✓	X
Archive / delete course	X	✓	X
Assign faculty to course	X	✓	X
Import from curriculum generator	✓	✓	X

Super admin is functionally a faculty with global scope. The UI should not diverge meaningfully between the two roles within a course. A super admin entering a course they do not teach should see a subtle banner indicating oversight mode, but controls remain enabled.

Faculty cannot self-enroll in a course; faculty assignment is a super admin action.

6. Design System

Extract these tokens into Tailwind config or CSS variables as the first dev task.

Colors:


```

--color-navy:           #132B5E    /* primary, nav background, emphasis text */
--color-navy-dark:      #0B1C42    /* hover / pressed */
--color-orange:         #F28C2C    /* accent, primary CTA, progress fills */
--color-orange-dark:    #D96F10    /* hover, warning text */
--color-cream:          #FAF6EE    /* app background */
--color-cream-dark:     #F1EADA    /* borders, dividers, hover backgrounds */
--color-ink:            #1A2540    /* primary text */
--color-muted:          #6B7280    /* secondary text, metadata */
--color-success:        #2F6D3C    /* completed state, present attendance */
--color-success-bg:     #E8F1E9
--color-warning:        #A35500    /* in-progress, needs attention */
--color-warning-bg:     #FDF0DE
--color-danger:         #B91C1C    /* missing submission, grading backlog */
--color-danger-bg:      #FEE2E2

```

Typography: Display font `Fraunces` (Google Fonts) for course titles, section headings, and large metric values. Body / UI font `Geist` (Google Fonts) for everything else. Both are loaded via `@import` in the prototype; production should self-host or use `next/font` equivalents for performance.

Spacing and radius: Cards use `rounded-xl` (12px) with `1px` border in `--color-cream-dark`. Buttons use `rounded-lg` (8px). Pills use `rounded-full`. Card interior padding ranges from `p-5` to `p-6` depending on density. Vertical rhythm between major blocks is `gap-6` (24px); within a block, `gap-4` or `gap-3`.

Component patterns: Primary CTA is orange solid with white text. Secondary CTA is navy solid with white text. Tertiary / neutral action is white with `--color-cream-dark` border. Destructive actions are red solid and should always require confirmation. Status pills follow a consistent shape: subtle background + darker text on the same hue, `px-2.5 py-1`, `text-xs font-medium`.

Elevation: Use borders, not shadows, as the default separation mechanism — shadows are reserved for overlays (modals, dropdowns). This matches the India Learns aesthetic already established and is more legible in the cream color system.

7. Screen Specifications

7.1 Dashboard (`/dashboard`)

Cross-course landing page. Four metric cards at the top (active courses, total students, awaiting grading, average attendance) showing values aggregated across all courses the faculty teaches. Below: a two-thirds-width "My courses" list with progress bars and "next session" indicator, and a

one-third-width "Today's schedule" panel showing sessions happening today with time, course, and location.

Clicking a course card navigates to `/courses/:id/content`. The next-session CTA deep-links directly to that session.

Empty states: No courses → "You haven't been assigned any courses yet. Contact your administrator." No sessions today → "Nothing on your schedule today."

7.2 Course Shell (`/courses/:id/*`)

Persistent header across all course tabs. Shows course code, term, student count, title, "View as student" button, and context-appropriate primary action. Six tabs: Overview, Content (default landing), Gradebook, Students, Announcements, Settings.

The primary action button in the header changes with the active tab: on Content it is "Add session"; on Gradebook it is "Export"; on Announcements it is "New announcement"; etc.

7.3 Content Tab (`/courses/:id/content`) — Primary View

The heart of the app. Displays a list of modules as collapsible cards. Each module card shows its number (M1, M2...), title, completion ratio, and a progress bar. Expanding a module reveals its sessions as a list.

Each session row shows status icon (check = completed, dot = in progress, empty circle = upcoming), session number, scheduled date, title, inline counts (materials, assignments, attendance taken yes/no, grading backlog if any), and status pill. The row is clickable and opens the session detail page.

At the bottom of each expanded module is a dashed "Add session to this module" button. At the top of the page: "Add module" (secondary) and "Add session" (primary, opens a modal asking which module).

Drag-and-drop reordering should be implemented using `@dnd-kit/core`. Modules reorder among themselves; sessions reorder within a module and can be moved between modules.

Empty state for a new course: A prominent two-option card — "Start from scratch" and "Import from curriculum generator" (see §9).

7.4 Session Detail (`/courses/:id/sessions/:id`)

Full-width header with back link, breadcrumb (course code, module name, session number), session title, date, time, location, "Edit session" and "Mark session complete" buttons.

Body is a two-column layout. Left column (2/3 width) contains three stacked cards: Description, Materials, Assignments. Right column (1/3 width) contains Attendance summary and Session notes.

The Materials card supports drag-and-drop file upload and shows each item with type icon, title, size, download button, and an overflow menu. The Assignments card lists assignments with submission progress bars and quick actions to view submissions or jump to grading.

The Attendance card shows a percentage, a stacked bar (present/late/absent), and a button to open the attendance roster as a modal or separate view. When attendance has not been taken, the pill reads "Pending" and the primary state of the card is a large "Take attendance" button.

The Notes textarea is private to faculty and super admins and auto-saves with a 2-second debounce.

7.5 Gradebook — Grid View (`/courses/:id/gradebook`)

Spreadsheet-style table. Rows are students, columns are assignments, plus attendance and computed average. Sticky first column (student name). Sticky header row. Cells show numeric grade with color coding: green ≥ 85 , default 70–84, orange < 70 . Missing submissions show a dash on warning background.

Clicking a grade cell opens an inline editor; clicking a student row navigates to the student profile; clicking an assignment column header switches to the per-assignment view filtered to that assignment.

Export produces a CSV with all visible data.

7.6 Gradebook — Per-assignment View (`/courses/:id/gradebook/assignments/:id`)

Focused view of a single assignment. Top panel shows assignment title, due date, and four summary metrics (submitted, needs grading, graded, missing). Below: a dropdown to switch assignments, a row of filter pills (All, Needs grading, Graded, Missing), and a table of submissions with student name, submitted timestamp, status pill, grade, and an action button.

Action button varies by status: "Grade & feedback" for `needs_grading`, "Remind student" for `missing`, "View" for `graded`. The grading action opens a full-screen modal or side panel with the submission files, a grading rubric, grade input, and feedback editor.

7.7 Overview Tab (`/courses/:id/overview`)

Quality-assurance view for self-monitoring (and super admin oversight). Four metric cards (course progress, average attendance, submission rate, grading backlog), a "Course structure at a glance" block visualizing each module as a segmented bar colored by session status, a "Needs attention" panel surfacing the three most urgent items, and a "Recent activity" feed showing the last five student and faculty actions.

The Needs attention items are opinionated — they link directly to the page that resolves them (grading backlog → gradebook filtered to `needs_grading`; attendance below threshold → student roster; missing materials → session detail).

7.8 Stubbed Tabs

Students, Announcements, and Settings render placeholder cards in v0.1. Design specs follow once Content, Session, Gradebook, and Overview are validated in production.

The Students tab will show a roster table with submission completeness, attendance rate, and cumulative grade per student, with click-through to individual profiles.

The Announcements tab reuses the existing production composer but adds a history list with read-receipt counts.

The Settings tab houses course metadata editing, grading scheme, attendance policy thresholds, collaborator management, and course archiving.

8. Key Behaviors & Business Logic

Session completion. Marking a session complete is an explicit action, not automatic. It requires attendance to have been taken and is confirmed with a dialog summarizing what is being locked (attendance becomes read-only, session appears in progress calculations). Completion can be undone within 7 days; after that it requires a super admin to reopen.

QA metric computation. All QA metrics are computed on read from the normalized data. Do not precompute and store them; staleness risk outweighs compute cost at this scale. Cache at the React Query layer with a 60-second `staleTime` to avoid hammering the backend.

Grading workflow is two-step. Saving a grade stores it as a draft visible only to faculty.

Publishing the grade makes it visible to the student. The gradebook grid shows draft grades with a subtle indicator (e.g. italic). This matters for fairness — a faculty member grading a batch of submissions wants to review consistency before any student sees their grade.

Attendance is taken per session, not per student. The faculty opens the session, sees the full roster, and marks each student present / absent / late. Defaults to present on first open to reduce clicks in the common case. Late is a distinct status, not a derived field.

Grading backlog is the faculty's primary pull-to-action. The count appears in four places: left nav ("To Grade" badge), dashboard metrics, course Overview panel, and inline on the session row in the Content tab. These should all reference the same underlying query `((submissions.status === 'needs_grading'))` scoped appropriately.

Materials vs. Assignments are conceptually distinct. Materials are things the student consumes. Assignments are things the student produces. A reading and a worksheet go in different buckets even if both relate to the same session.

This section documents the integration plan between the LMS and the team's existing curriculum generator, located at github.com/viditkbhatnagar/CurriculumGenerator. The repo has been inspected and the integration contract below reflects the generator's actual architecture and data model — not guesses based on its output documents.

9.1 The Curriculum Generator — What It Actually Is

The curriculum generator is a monorepo Node.js / Express / MongoDB application with a Next.js frontend. It is an AI-powered course design tool that runs a course from a Subject Matter Expert's initial input through 13 sequential "steps", each producing a structured output that is persisted to MongoDB and approved before the next step runs. It also produces downloadable Word, PDF, and SCORM exports at the end. Source documents like the Airline & Airport Management `.docx` are a *rendering* of that structured data, not the data itself.

This matters for the integration: **the generator already has a rich, structured backend that the LMS should consume directly.** Parsing exported `.docx` files would throw away most of the value and introduce fragility.

9.2 The 13-Step Workflow — Which Steps Matter for the LMS

The full workflow has 13 steps. Not all of them are relevant to the LMS. The table below summarizes each step and its relevance to the import flow.

Step	Output	Relevance to LMS
1	Program overview (aim, duration, target audience, career outcomes)	Course metadata
2	Competency framework (KSA — Knowledge, Skills, Attitudes)	Reference data, stored as Course or Program metadata
3	Program Learning Outcomes (PLOs)	Course.programLearningOutcomes
4	Course framework — modules with hours, MLOs, activities	Module skeleton + MLOs
5	Topic-level sources (APA-cited references per MLO)	Reference material for Step 10 readings
6	Module reading lists (core + supplementary, with estimated minutes)	Material entries of type reading
7	(reserved / streaming implementation detail)	—
8	Additional validation / research	Internal to generator
10	Lesson plans — per-lesson breakdown with activities, timing, materials, instructor notes	Session entities with full detail
11	PPT decks generated per lesson (.pptx + .pdf files)	Material entries of type slides , with stored file references
12	Assignment packs (briefs, rubrics, evidence requirements)	Assignment entities with rubrics
13	Summative exam (MCQ + case + scenario sections, marking scheme)	Final Assignment with type exam

Steps 4, 10, 11, 12, and 13 are the load-bearing ones for the LMS. Step 4 gives the module skeleton and MLOs. Step 10 is the crown jewel — it contains the lesson-by-lesson breakdown that maps directly to LMS sessions, including lesson duration, linked MLOs, ordered activities with types and durations, required materials (PPT, case files, readings), instructor pedagogical notes, independent study assignments, and formative check questions. Step 11 adds the actual slide files. Step 12 adds assignment briefs with rubrics. Step 13 adds the final exam.

9.3 Data Model Mapping — Generator → LMS

This is the authoritative mapping. The left column uses the generator's MongoDB field names (from `packages/backend/src/models/`); the right column uses the LMS types defined in §4.

Program → Course

Generator field	LMS field	Notes
<code>Program.programName</code>	<code>Course.title</code>	
<code>Program.qualificationLevel</code> + <code>qualificationType</code>	<code>Course.description</code> (prefix) or dedicated metadata field	e.g. "Level 5 Diploma"
Step 1 <code>programOverview.duration</code>	<code>Course.totalHours</code>	Parse numeric portion
Step 4 <code>totalContactHours</code>	<code>Course.contactHours</code>	
Step 4 <code>totalIndependentHours</code>	<code>Course.independentHours</code>	
Step 1 <code>programOverview.aim</code>	<code>Course.description</code>	

Step 4 Module → LMS Module

Generator field	LMS field
<code>modules[].moduleCode</code>	<code>Module.code</code> (new field — add to §4)
<code>modules[].title</code>	<code>Module.title</code>
<code>modules[].sequenceOrder</code>	<code>Module.order</code>
<code>modules[].totalHours</code>	<code>Module.totalHours</code>
<code>modules[].contactHours</code>	<code>Module.contactHours</code>
<code>modules[].independentHours</code>	<code>Module.independentHours</code>
<code>modules[].isCore</code>	<code>Module.coreElective</code> ('core' / 'elective')
<code>modules[].mlos[]</code>	<code>Module.learningOutcomes[]</code> — all fields map directly
<code>modules[].mlos[].bloomLevel</code>	<code>ModuleLearningOutcome.bloomLevel</code>
<code>modules[].mlos[].linkedPLOs</code>	<code>ModuleLearningOutcome.linkedPLOs</code> (add to §4)
<code>modules[].mlos[].competencyLinks</code>	<code>ModuleLearningOutcome.linkedKSCs</code>

Step 10 Lesson → LMS Session

This is the most important mapping. Each `LessonPlan` in `step10.moduleLessonPlans[].lessons[]` becomes a `Session` in the LMS.

Generator field	LMS field
<code>lessons[].lessonId</code>	<code>Session.sourceLessonId</code> (new field — tracks origin)
<code>lessons[].lessonNumber</code>	<code>Session.number</code>
<code>lessons[].lessonTitle</code>	<code>Session.title</code>
<code>lessons[].duration</code> (minutes)	<code>Session.plannedMinutes</code> (new field — was <code>plannedHours</code> , rename for precision)
<code>lessons[].objectives[]</code>	<code>Session.description</code> (rendered as objectives list)
<code>lessons[].linkedMLOs</code>	<code>Session.linkedMLOs</code> (new field — array of MLO IDs)
<code>lessons[].activities[]</code>	Rendered inside <code>Session.description</code> or a structured <code>Session.activities[]</code> field
<code>lessons[].instructorNotes</code>	<code>Session.notes</code> (private)
<code>lessons[].materials.pptDeckRef</code>	<code>Material</code> entry of type <code>slides</code>
<code>lessons[].materials.caseFiles[]</code>	<code>Material</code> entries of type <code>file</code>
<code>lessons[].materials.readingReferences[]</code>	<code>Material</code> entries of type <code>reading</code>
<code>lessons[].independentStudy.coreReadings</code>	<code>Material</code> entries of type <code>reading</code> , marked as core
<code>lessons[].formativeChecks[]</code>	<code>Session.formativeChecks</code> (new field) or rendered inline
<code>lessons[].caseStudyActivity</code>	Rendered in session description; linked <code>Assignment</code> if <code>activityType === 'assessment_ready'</code>

Step 11 PPT → Material

Each `moduleLessonPlans[].pptDecks[]` entry references a generated `.pptx` and `.pdf` file by path. The LMS should either copy these files into its own storage on import, or store the

generator's file references and proxy access. Given the generator uses Render Persistent Disk and the LMS will have its own file storage, **copy on import** is the safer default.

Step 12 AssignmentPack → LMS Assignment

Generator field	LMS field
<code>AssignmentPack.assignmentBrief.title</code>	<code>Assignment.title</code>
<code>AssignmentPack.assignmentBrief.scenario</code> + <code>tasks</code>	<code>Assignment.description</code>
<code>AssignmentPack.rubric[]</code>	<code>Assignment.rubric[]</code>
<code>AssignmentPack.evidenceRequirements[]</code>	<code>Assignment.submissionRequirements</code> (new field)
<code>AssignmentPack.assessedLearningOutcomes[]</code>	<code>Assignment.linkedMLOs</code> (new field)

Each assignment pack is tied to a module in the generator, not a specific lesson. The LMS should attach it to the final session in the module by default, and allow faculty to reassign to a different session in the pre-import preview.

Step 13 SummativeExam → LMS Assignment

The summative exam is a single `Assignment` tied to the final session of the course, with `type: 'exam'`. Its `sectionA/B/C` structure and `markingScheme` become the assignment description and rubric.

9.4 Integration Approach — API, Not File Upload

The user's initial framing was that faculty would "upload the raw materials created from the curriculum generator" and the LMS would auto-populate from those files. After inspecting the generator, this should be reconsidered. The generator stores everything in MongoDB with a clean REST API — the `.docx` is an export format for human review, not the source of truth.

Parsing `.docx` would be building a second, inferior pipeline on top of the real one.

Primary path: API integration. The LMS calls the generator's API directly, fetches the structured workflow data, and imports it. The generator exposes:

GET /api/workflow/:workflowId	→ full workflow document
GET /api/workflow/:workflowId/progress	→ status and progress
GET /api/workflow/:workflowId/step4	→ module structure (implicit from full doc)
GET /api/workflow/:workflowId/step10	→ lesson plans
GET /api/workflow/:workflowId/step10/status	→ polling endpoint for long-running generation
GET /api/workflow/:workflowId/export/word	→ legacy .docx export
GET /api/workflow/:workflowId/export/pdf	→ legacy PDF export
GET /api/workflow/:workflowId/export/scorm	→ SCORM package export

The import flow in the LMS therefore becomes: faculty pastes a workflow ID or URL from the generator → LMS calls the generator API → LMS shows the preview → faculty confirms → LMS persists as its own entities.

Fallback path: .docx upload. Kept for edge cases — SMEs who generated a course in the generator but export-and-archive, or offline workflows, or legacy documents. This should be a secondary UI option with a clear disclaimer that some richness (lesson activity breakdowns, rubrics, formative checks) will be lost vs. the API path because the `.docx` does not render every field.

Authentication. The generator uses Auth0. The LMS should use a dedicated service account with read-only scope to call the generator's API, or — if the product team permits — accept the user's Auth0 token via SSO and call the generator on their behalf. The latter is cleaner but requires both systems to trust the same Auth0 tenant.

9.5 Sync Model — One-Time Import, Not Live Link

Three possible sync models were considered. Recommendation below.

Option A — One-time import (recommended). LMS fetches once, persists its own copy, and the two systems are decoupled from that point. Faculty edits in the LMS are authoritative and do not flow back to the generator. Pros: simple, safe, no coupling risk. Cons: if the SME revises the curriculum in the generator, the LMS version is stale.

Option B — Live link. LMS stores only a reference and fetches fresh data on every load. Pros: always current. Cons: LMS is unusable if the generator is down; faculty edits conflict with upstream changes; performance penalty; complex permission model across two systems. Not recommended.

Option C — Hybrid with refresh. LMS imports once but stores the source workflow ID and offers a "Re-sync from generator" action. Faculty chooses when to pull fresh data. Any local edits get flagged for conflict resolution. This is the best long-term pattern but adds substantial complexity.

Recommendation: ship Option A for v1, design data model to support Option C later.

Specifically, add `Course.sourceWorkflowId` and `Course.sourceWorkflowVersion` fields at the start so a re-sync feature can be layered in without migration.

9.6 Minimum Integration Contract

These endpoints / behaviors must exist on the generator side for API integration to work. Some already do; others need confirmation or may need new work by the generator's maintainer.

Required from the generator:

1. A stable, documented API endpoint that returns a completed workflow as JSON — ideally `GET /api/workflow/:id` returning the full workflow document, or a dedicated `GET /api/workflow/:id/lms-export` that returns a pre-shaped payload matching the LMS's import schema.
2. Authentication method the LMS can use: either service-account API keys or a shared Auth0 tenant.
3. File access for generated PPT decks and case files — either signed URLs the LMS can download from, or direct file downloads via the API.
4. A status flag indicating the workflow is "complete" — i.e., steps 10, 11, 12, 13 have all been generated and approved. The LMS should refuse to import incomplete workflows.
5. A stable schema versioning policy — if the generator's internal models change, the LMS import must not break silently. A `schemaVersion` field on the returned payload is the minimum; a changelog / migration notice is better.

Required from the LMS (this codebase):

1. A new `CurriculumImport` service that calls the generator API, validates the response against a schema, transforms it into LMS entities, and persists them transactionally (all or nothing — a half-imported course is worse than no import).
2. A schema validator (e.g. Zod) for the expected generator payload, so unexpected shape changes fail loudly with a clear error message.
3. Storage handling for imported files (copy from generator's file URLs into LMS storage).
4. Audit logging of every import (who imported, from what workflow ID, when, what entities were created).

9.7 Faculty UX for Import

The empty-state card on a new course's Content tab offers three options:

1. **Import from Curriculum Generator** (primary) — opens a modal asking for either a generator URL, a workflow ID, or (if SSO is configured) a picker showing the faculty's own completed workflows in the generator.
2. **Upload generated document** (secondary) — `.docx` fallback for legacy cases.
3. **Start from scratch** (tertiary) — existing manual flow.

After fetching, the LMS shows a full preview screen: course metadata at the top, a collapsible tree of every module, session, assignment, and material that will be created, with edit-in-place for titles and a remove button on each item. Parsing warnings appear inline (e.g., "Session 4 has no PPT deck in the generator output").

Below the tree: a bulk-schedule panel where faculty picks a start date and a meeting cadence (Mondays & Wednesdays at 2pm, or a custom pattern) and the LMS auto-fills session dates.

Confirm button is labeled "Import N modules, M sessions, K assignments" so the faculty knows exactly what they are committing to. Clicking it creates everything in one transaction.

Post-import, a banner on the course reads: "Imported from Curriculum Generator on [date]. Last synced: [date]." This sets up the Option C re-sync feature for later.

9.8 Open Questions for the Generator Maintainer

These need answers from the generator's team before integration is built. Tag `@viditkbhatnagar` on these.

1. **Is there already a dedicated LMS export endpoint, or should we fetch and transform the raw workflow document?** If the former, what is its schema? If the latter, is the `CurriculumWorkflow` schema in `packages/backend/src/models/CurriculumWorkflow.ts` stable enough to consume directly?
2. **Auth model.** Does the generator support service-account API keys, or is Auth0 JWT the only option? If Auth0, can the LMS and generator share a tenant, or will each user need separate credentials?
3. **File storage access.** PPT decks and case files are referenced by path in the workflow document. Can the generator expose a signed-URL endpoint so the LMS can download these, or should the LMS scrape them from Render Persistent Disk directly? (The latter is bad — please don't.)
4. **Completion status.** Is there a single `workflow.status === 'complete'` flag, or does the LMS need to check each of steps 10, 11, 12, 13 independently? What does "approved" mean in this context — is an unapproved-but-generated Step 10 safe to import?

5. **Schema stability.** The `CurriculumWorkflow` schema has evolved across versions (evidence: `STEP10_*` fix summaries throughout the repo). What is the version number of the current schema, and what is the policy on breaking changes? Can the LMS pin to a specific version?
6. **Re-generation semantics.** If an SME re-runs Step 10 after an LMS import, the lesson IDs in the generator may change. How stable are `lessonId` values across re-generations? If they are not stable, the LMS cannot rely on them as foreign keys for future re-sync.
7. **Partial programs.** Many diploma programs in the generator are Level 5 / 7 / multi-year. Does a "program" in the generator map to one LMS course, or to multiple (one per academic year)? The LMS currently assumes one program = one course.
8. **Assignment-to-session binding.** Step 12 assignment packs are module-scoped, not lesson-scoped. Is there any structured hint in the generator about which lesson an assignment naturally pairs with (e.g. due week, linked activity), or should the LMS default to "attach to final session of module"?
9. **MLO visibility policy.** Does the generator intend MLOs to be student-visible? The LMS needs a policy — show to students, or faculty-only?
10. **Accreditation and KSC registry.** The KSC codes (K1, S3, C2, ...) in the MLOs are from a competency framework (Step 2). Is this framework an institutional constant, or per-program? Should the LMS store KSCs as a first-class entity shared across courses, or as free-text per-MLO tags?

9.9 Recommended Build Sequence

1. Reach out to the generator maintainer with §9.8 questions. Do not start build until at least the auth model, completion flag, and schema stability answers are known.
2. Build a schema validator (Zod) for the expected `CurriculumWorkflow` payload.
3. Build the transformation layer (generator payload → LMS entities) as a pure function, fully unit-tested against fixtures. Use the generator's own `demo_curriculum/` samples as fixtures.
4. Build the API client with retry and circuit-breaker behavior.
5. Build the preview UI.
6. Build the transactional persistence layer.
7. Ship behind a feature flag, enabled for one pilot faculty member. Compare imported-course-quality against manually-built-course-quality.
8. Iterate, then enable broadly.
9. Add `.docx` fallback as a separate feature after the API path is proven.

10. States to Handle

Every screen needs to handle these states explicitly — the prototype shows the happy path only.

Loading. Skeleton screens matching the final layout, not spinners. Skeleton duration >200ms to avoid flash.

Empty. First-time courses (no modules), newly-created modules (no sessions), sessions with no materials, assignments with no submissions, courses with no enrolled students. Each deserves a friendly empty state that also shows the next action.

Error. Network failure, permission denied, stale data conflicts (two faculty editing the same session). Show inline error with retry where sensible, not full-page error screens.

Permission-variant. The same screen rendered for a super admin in oversight mode vs. the assigned faculty. Minimal visual difference but a clear oversight banner at the top.

Stale data. Attendance marked while a student is being removed from the roster. Grade submitted after an assignment is deleted. Handle gracefully; never corrupt the database state.

Archived courses. Read-only mode for archived courses. All edit controls hidden, not merely disabled.

11. Out of Scope for v0.1

Video conferencing integration (this is an in-person LMS).

Plagiarism / originality checking.

Real-time collaborative editing of materials.

Mobile-native apps (the web UI should be responsive but v0.1 is desktop-first).

Parent / guardian portals.

Peer assessment, group assignments.

Advanced rubrics with weighted criteria (simple free-text rubrics only in v0.1).

Cross-course analytics beyond the dashboard metrics (full analytics dashboard is a future phase).

12. Open Questions (LMS Scope)

Curriculum generator integration questions are consolidated in §9.8. Remaining open questions

about LMS behavior:

Attendance threshold. The Overview tab flags students "below 80% attendance." Is 80% the institutional standard? Should this be configurable per course?

Grade visibility timing. When a faculty publishes a grade, should the student be notified immediately or should there be a batch / scheduled release option?

Co-teaching. The data model supports multiple faculty per course, but the UI does not address conflicts (two faculty marking the same attendance simultaneously, grading the same submission). How common is co-teaching in practice, and how rigorous does conflict handling need to be?

Session rescheduling. What happens to attendance and completion status when a session is rescheduled? Can a completed session be moved?

Assignment late policy. Should late submissions be flagged differently from on-time submissions in the gradebook? Is there an automatic late penalty?

Student profile depth. The `(students/:studentId)` route is in the IA but not specified. What does a faculty need to see there — just grades and attendance for this course, or a cross-course view of the student?

Appendix: Prototype Reference

The interactive prototype (`(india_learns_faculty_lms.jsx)`) is the source of truth for visual design, spacing, and interaction patterns. When this document and the prototype disagree, prioritize the prototype for pixel-level questions and this document for data and behavior.

Mock data in the prototype (course "Digital Marketing Fundamentals", faculty "Prof. Priya Raman", student names) should not ship to production. Use the curriculum generator's Airline and Airport Management course as the first real dataset for integration testing.